Lesson 02 Demo 04

Implementing Blue/Green Deployment Using CodeDeploy

Objective: To implement a blue/green deployment strategy using AWS CodeDeploy and gain proficiency in setting up a continuous integration pipeline using AWS CodePipeline and CodeBuild for automating Maven-based build and test processes

Tools required: AWS CodeBuild, AWS CodePipeline, and AWS Elastic Beanstalk

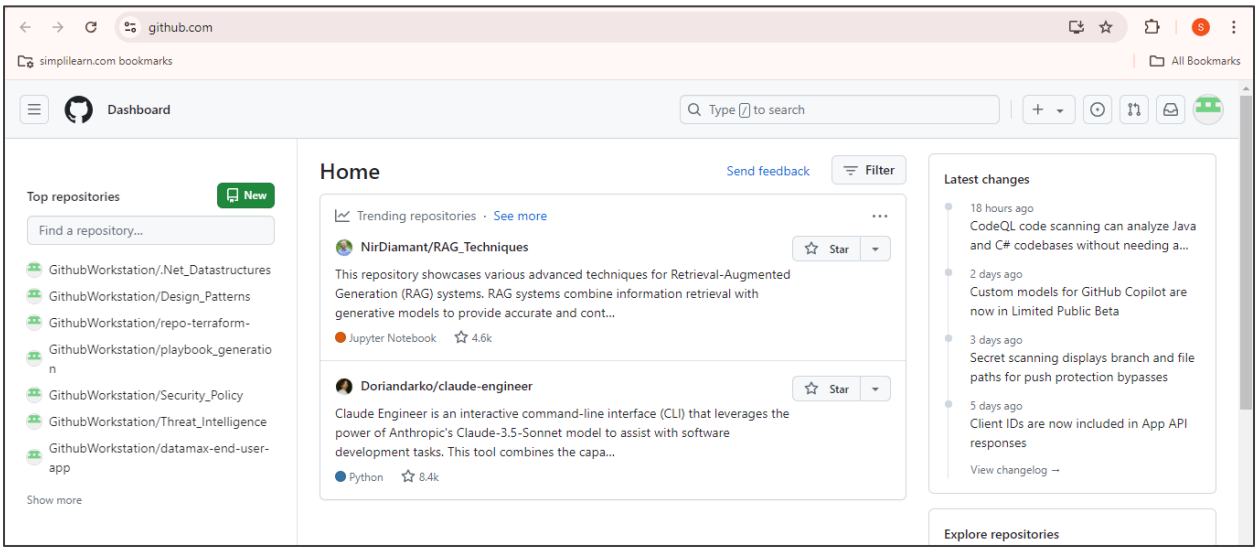
Prerequisites: Deployment of a sample application on AWS Elastic Beanstalk

Steps to be followed:

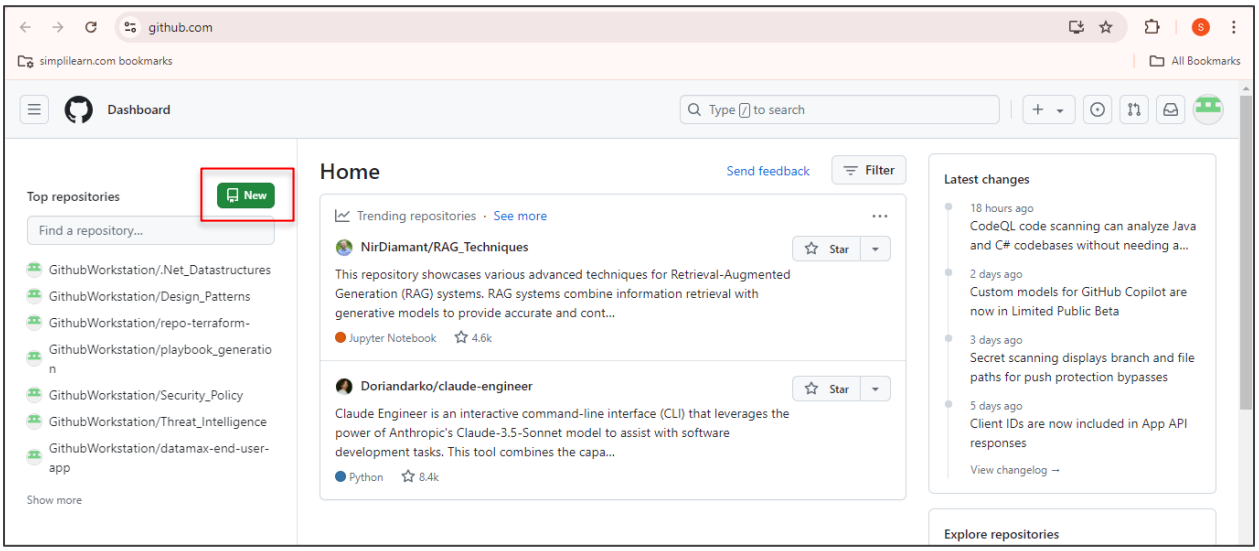
1. Import the repository from GitHub
2. Configure CodeBuild and CodePipeline to perform build and test automation

Step 1: Import the repository from GitHub

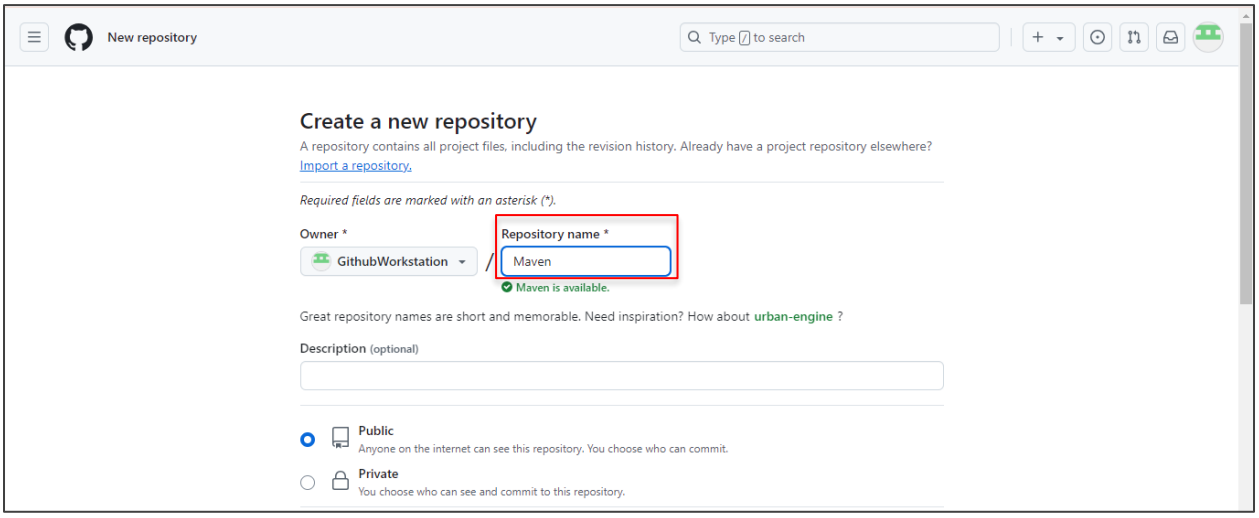
1.1 Go to the following URL and sign in to your GitHub account:
https://github.com/

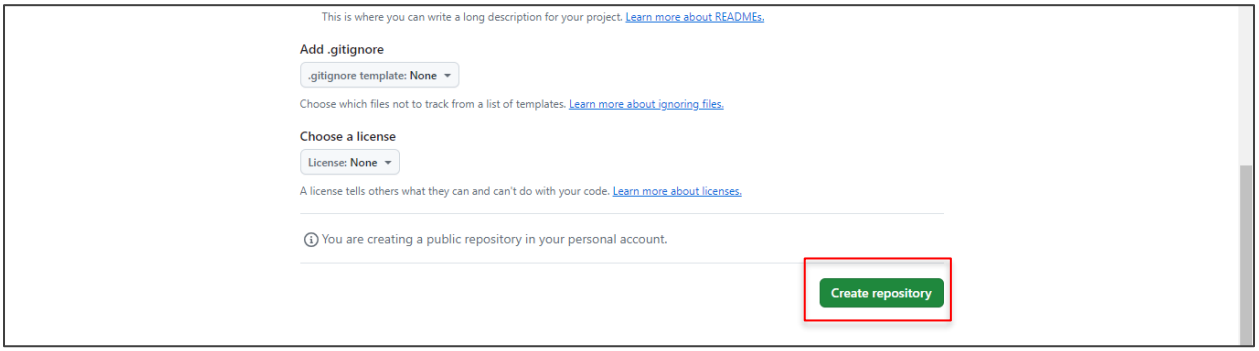


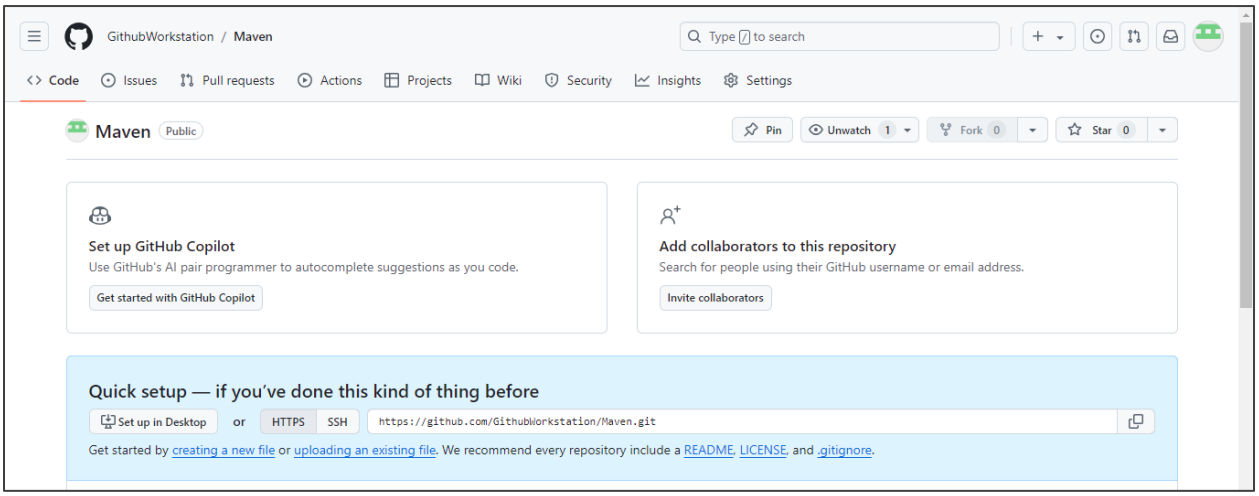
1.2 Click on the **New** button to create a new repository



1.3 Name the repository as **Maven**, scroll down, and click on **Create repository**

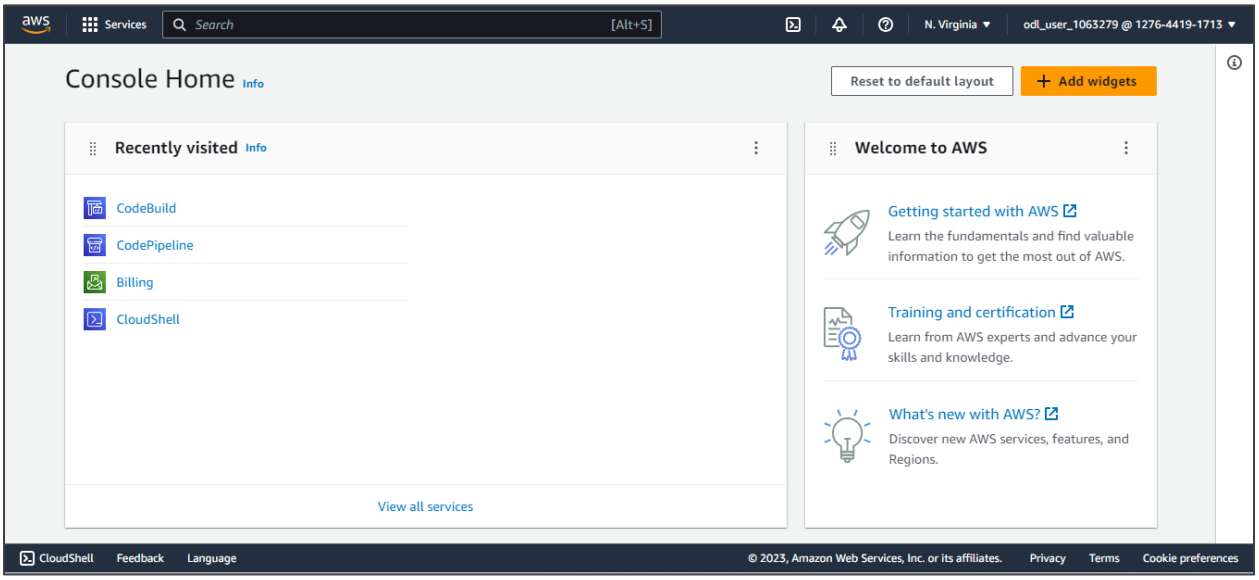




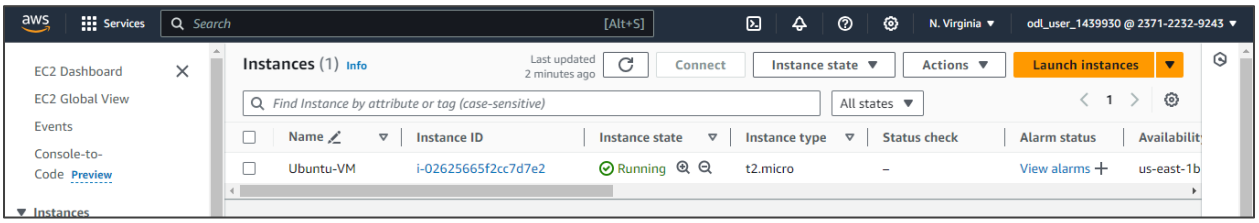


The repository is created successfully.

1.4 Sign in to the AWS Console



1.5 Create an EC2 instance and connect to it



1.6 Create a directory using the following command:
mkdir demo

```
ubuntu@ip-172-31-93-195:~$ mkdir demo
ubuntu@ip-172-31-93-195:~$
```

1.7 Navigate to the created directory using the following command:
cd demo

```
ubuntu@ip-172-31-93-195:~$ cd demo
ubuntu@ip-172-31-93-195:~/demo$
```

1.8 Initialize Git using the following command:
git init

```
ubuntu@ip-172-31-93-195:~/demo$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/ubuntu/demo/.git/
ubuntu@ip-172-31-93-195:~/demo$
```

1.9 Clone the GitHub repository using the following command:
git clone --mirror https://github.com/anujdevopslearn/MavenBuild MavenBuild

```
ubuntu@ip-172-31-93-195:~/demo$ git clone --mirror https://github.com/anujdevopslearn/MavenBuild MavenBuild
Cloning into bare repository 'MavenBuild'...
remote: Enumerating objects: 1986, done.
remote: Counting objects: 100% (381/381), done.
remote: Compressing objects: 100% (153/153), done.
remote: Total 1986 (delta 282), reused 274 (delta 212), pack-reused 1605 (from 1)
Receiving objects: 100% (1986/1986), 444.03 KiB | 11.38 MiB/s, done.
Resolving deltas: 100% (727/727), done.
ubuntu@ip-172-31-93-195:~/demo$
```

1.10 Switch to the cloned repository using the following command:
cd MavenBuild/

```
ubuntu@ip-172-31-93-195:~/demo$ cd MavenBuild/
ubuntu@ip-172-31-93-195:~/demo/MavenBuild$
```

1.11 List all files present in the repository using the following command:
ls -lart

```
ubuntu@ip-172-31-93-195:~/demo/MavenBuild$ ls -lart
total 44
drwxrwxr-x 4 ubuntu ubuntu 4096 Aug 30 06:39 ..
drwxrwxr-x 4 ubuntu ubuntu 4096 Aug 30 06:39 refs
drwxrwxr-x 2 ubuntu ubuntu 4096 Aug 30 06:39 info
drwxrwxr-x 2 ubuntu ubuntu 4096 Aug 30 06:39 hooks
-rw-rw-r-- 1 ubuntu ubuntu   73 Aug 30 06:39 description
drwxrwxr-x 2 ubuntu ubuntu 4096 Aug 30 06:39 branches
drwxrwxr-x 4 ubuntu ubuntu 4096 Aug 30 06:39 objects
-rw-rw-r-- 1 ubuntu ubuntu  176 Aug 30 06:39 config
-rw-rw-r-- 1 ubuntu ubuntu 3344 Aug 30 06:39 packed-refs
-rw-rw-r-- 1 ubuntu ubuntu   23 Aug 30 06:39 HEAD
drwxrwxr-x 7 ubuntu ubuntu 4096 Aug 30 06:39 .
ubuntu@ip-172-31-93-195:~/demo/MavenBuild$
```

1.12 List all remote repositories using the following command:
git remote -v

```
ubuntu@ip-172-31-93-195:~/demo/MavenBuild$ git remote -v
origin  https://github.com/anujdevopslearn/MavenBuild (fetch)
origin  https://github.com/anujdevopslearn/MavenBuild (push)
ubuntu@ip-172-31-93-195:~/demo/MavenBuild$
```

1.13 Remove the existing remote configuration using the following command:
git remote remove origin

```
ubuntu@ip-172-31-93-195:~/demo/MavenBuild$ git remote remove origin
Note: Some branches outside the refs/remotes/ hierarchy were not removed;
to delete them, use:
  git branch -d develop
  git branch -d master
ubuntu@ip-172-31-93-195:~/demo/MavenBuild$
```

1.14 Add a new remote repository using the following command:
git remote add origin Repository-link

```
ubuntu@ip-172-31-93-195:~/demo/MavenBuild$ git remote add origin https://github.com/GithubWorkstation/Maven.git
ubuntu@ip-172-31-93-195:~/demo/MavenBuild$
```

Note: Replace **Repository-link** with the link to the repository you created in Step 1.3

1.15 Verify if the new remote is correctly configured using the following command:
git remote -v

```
ubuntu@ip-172-31-93-195:~/demo/MavenBuild$ git remote -v
origin  https://github.com/GithubWorkstation/Maven.git (fetch)
origin  https://github.com/GithubWorkstation/Maven.git (push)
```

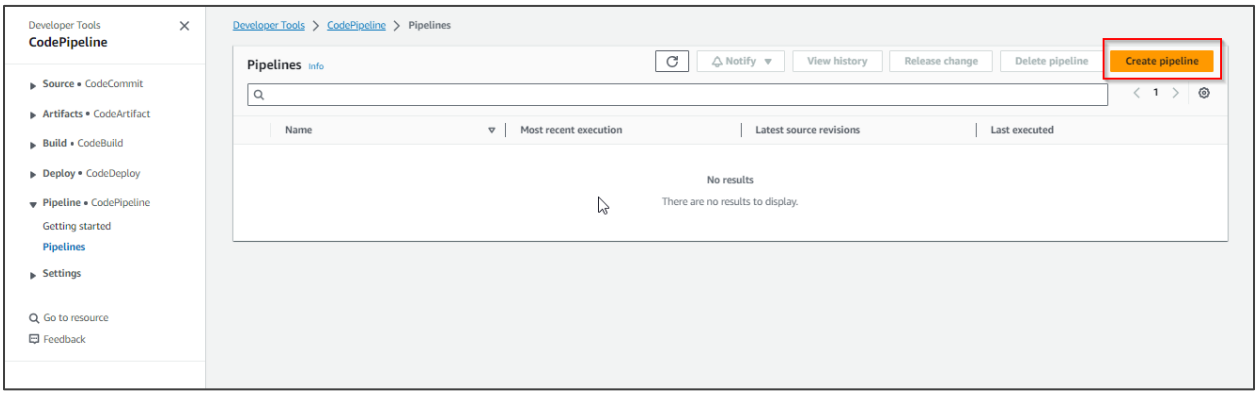
1.16 Upload your local repository content to the remote repository using the following command:
git push --all origin

```
ubuntu@ip-172-31-93-195:~/demo/MavenBuild$ git push --all origin
Username for 'https://github.com': GithubWorkstation
Password for 'https://GithubWorkstation@github.com':
Enumerating objects: 1606, done.
Counting objects: 100% (1606/1606), done.
Compressing objects: 100% (765/765), done.
Writing objects: 100% (1606/1606), 368.63 KiB | 36.86 MiB/s, done.
Total 1606 (delta 560), reused 1589 (delta 546), pack-reused 0
remote: Resolving deltas: 100% (560/560), done.
To https://github.com/GithubWorkstation/Maven.git
 * [new branch]      develop -> develop
 * [new branch]      master -> master
ubuntu@ip-172-31-93-195:~/demo/MavenBuild$
```

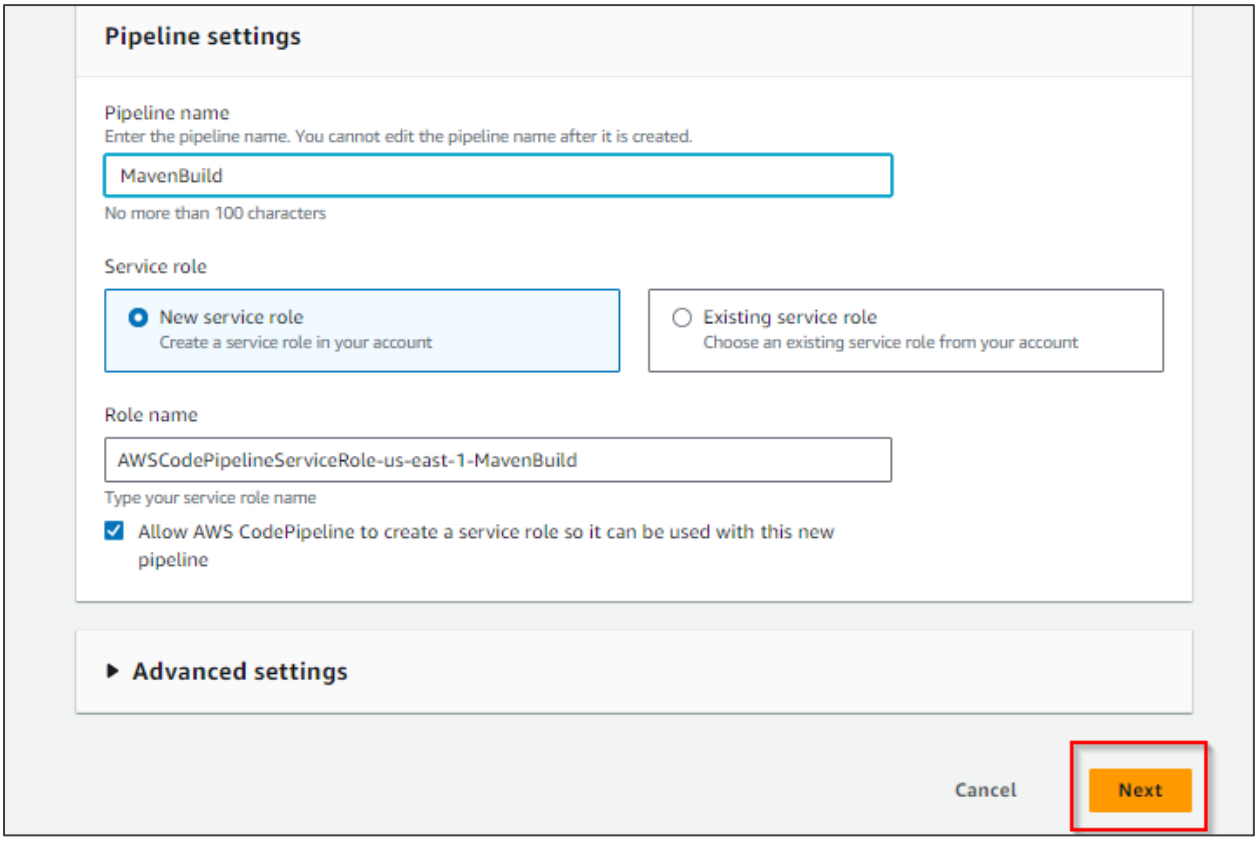
Note: When prompted, enter your GitHub credentials

Step 2: Configure CodeBuild and CodePipeline to perform build and test automation

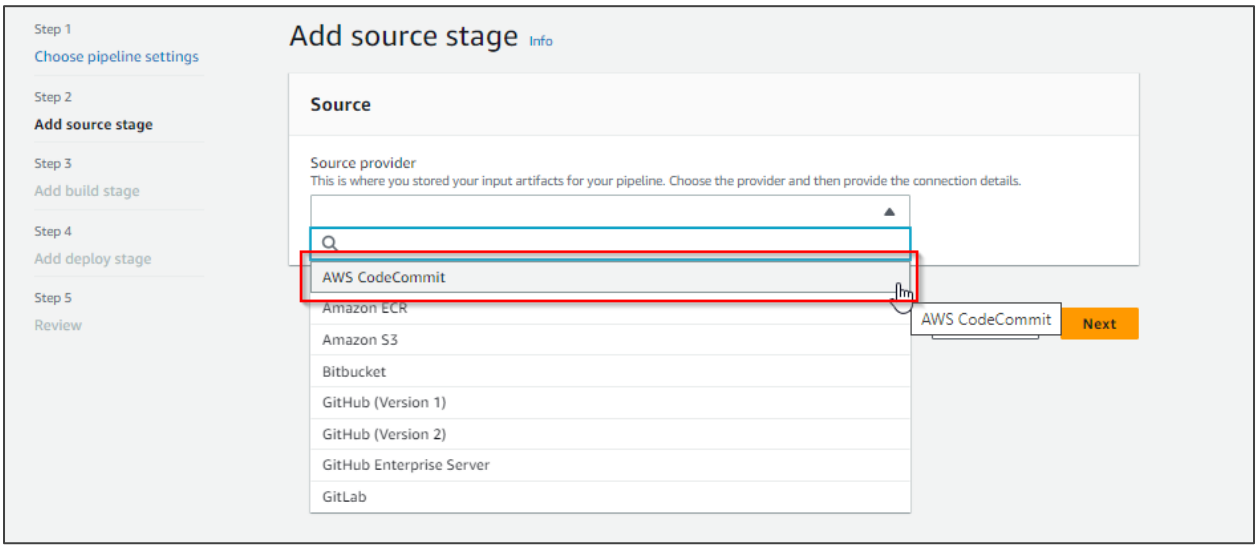
2.1 Click on the **Create pipeline** button



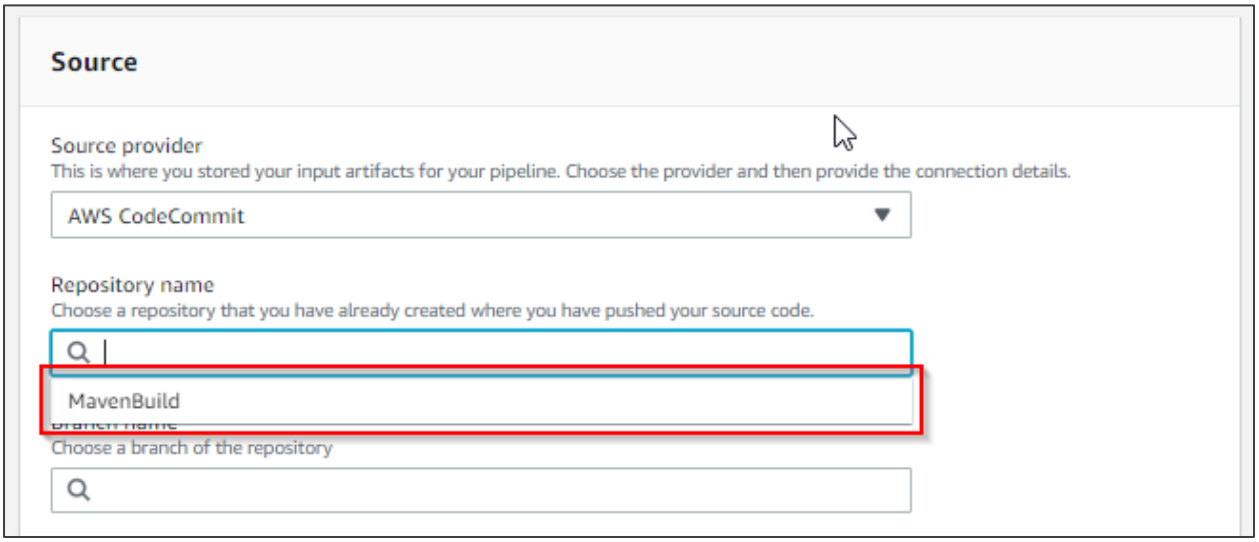
2.2 Add a name for the pipeline and click on the **Next** button



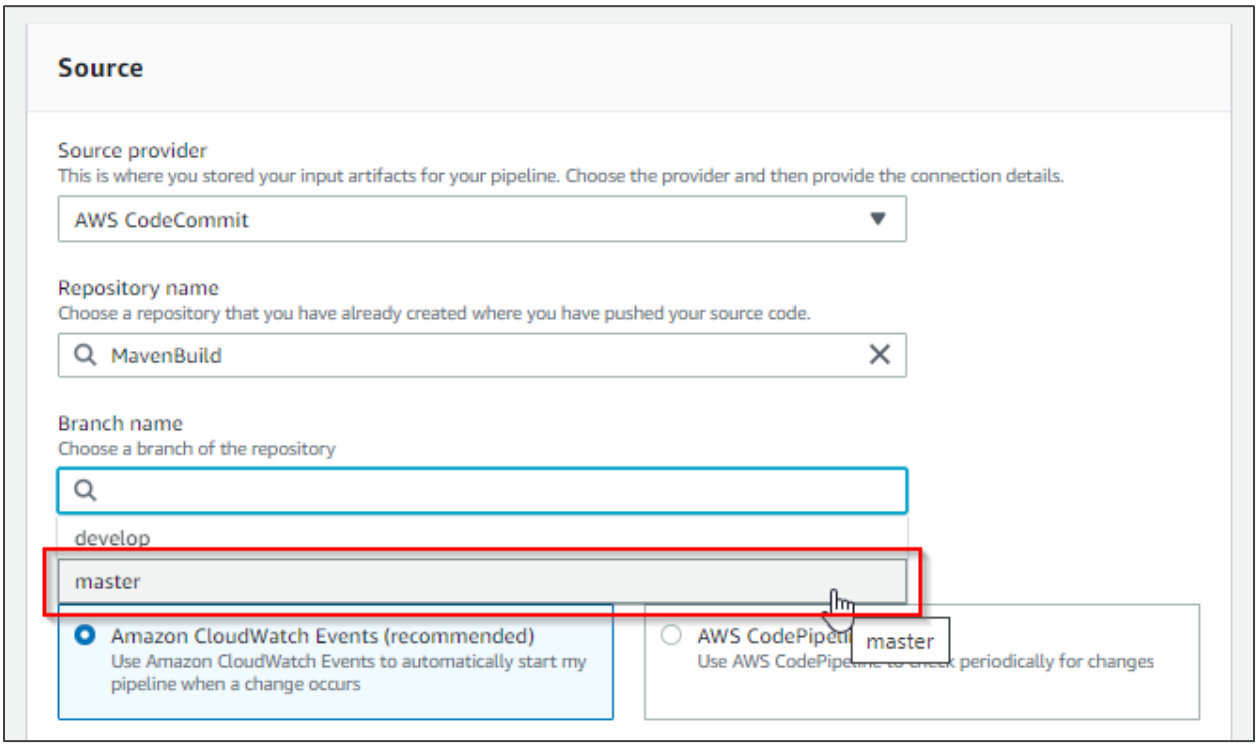
2.3 Under **Source provider**, select **AWS CodeCommit**



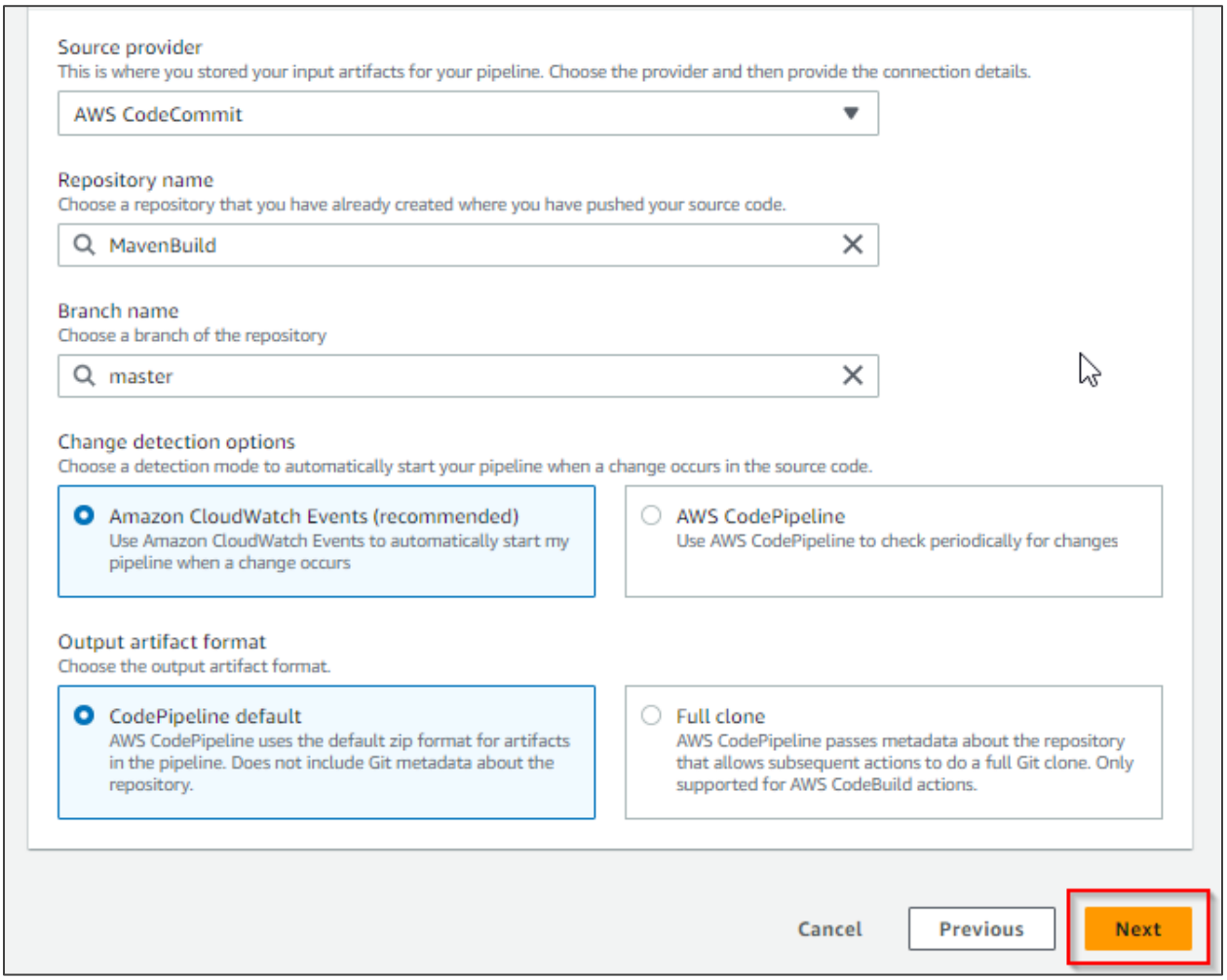
2.4 Select **MavenBuild** as the **Repository name**



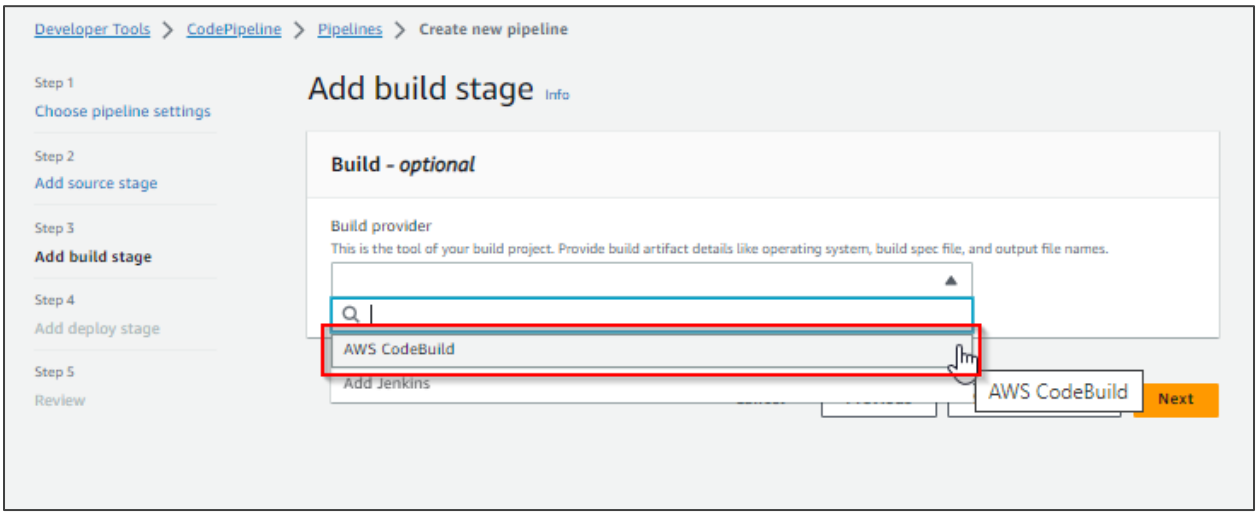
2.5 Select **master** as the **Branch name**



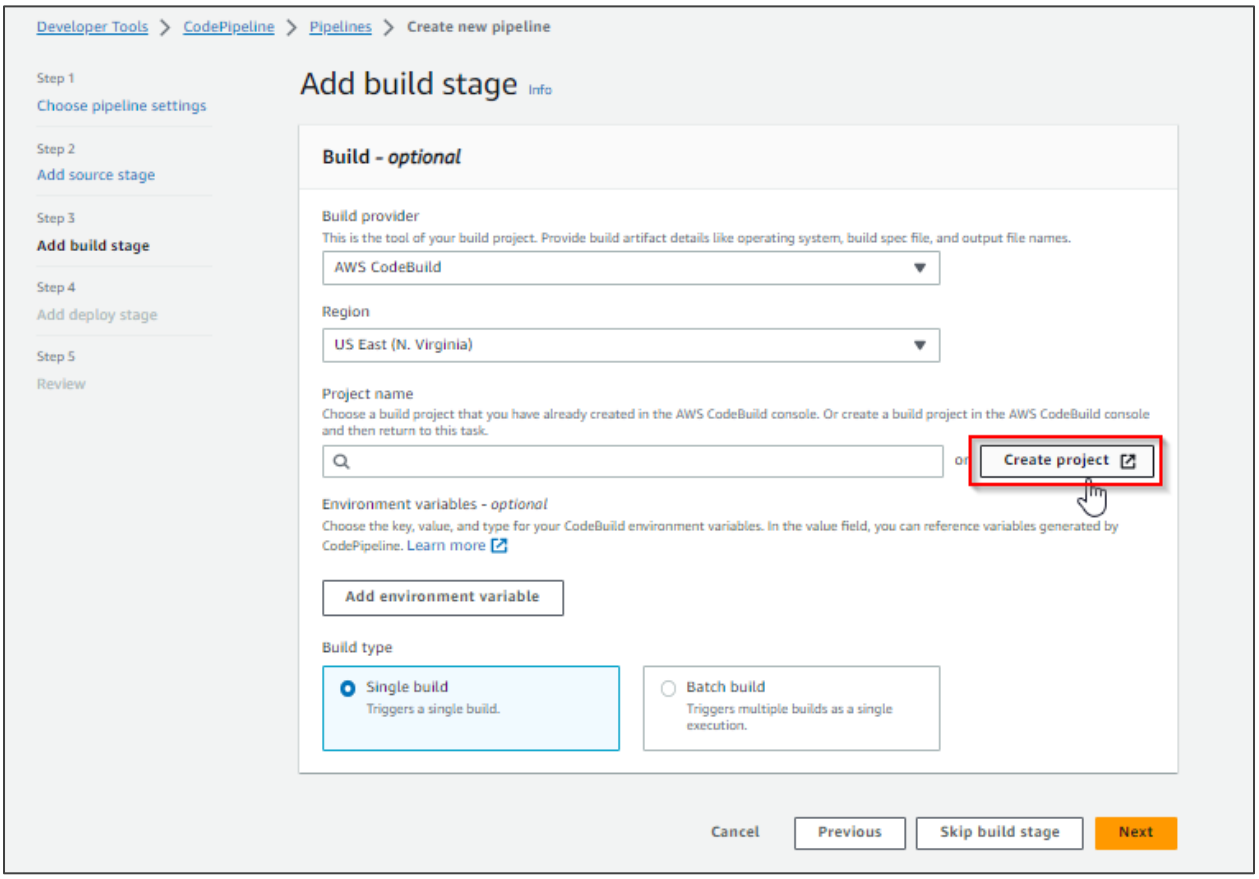
2.6 Once the details are entered, click on the **Next** button



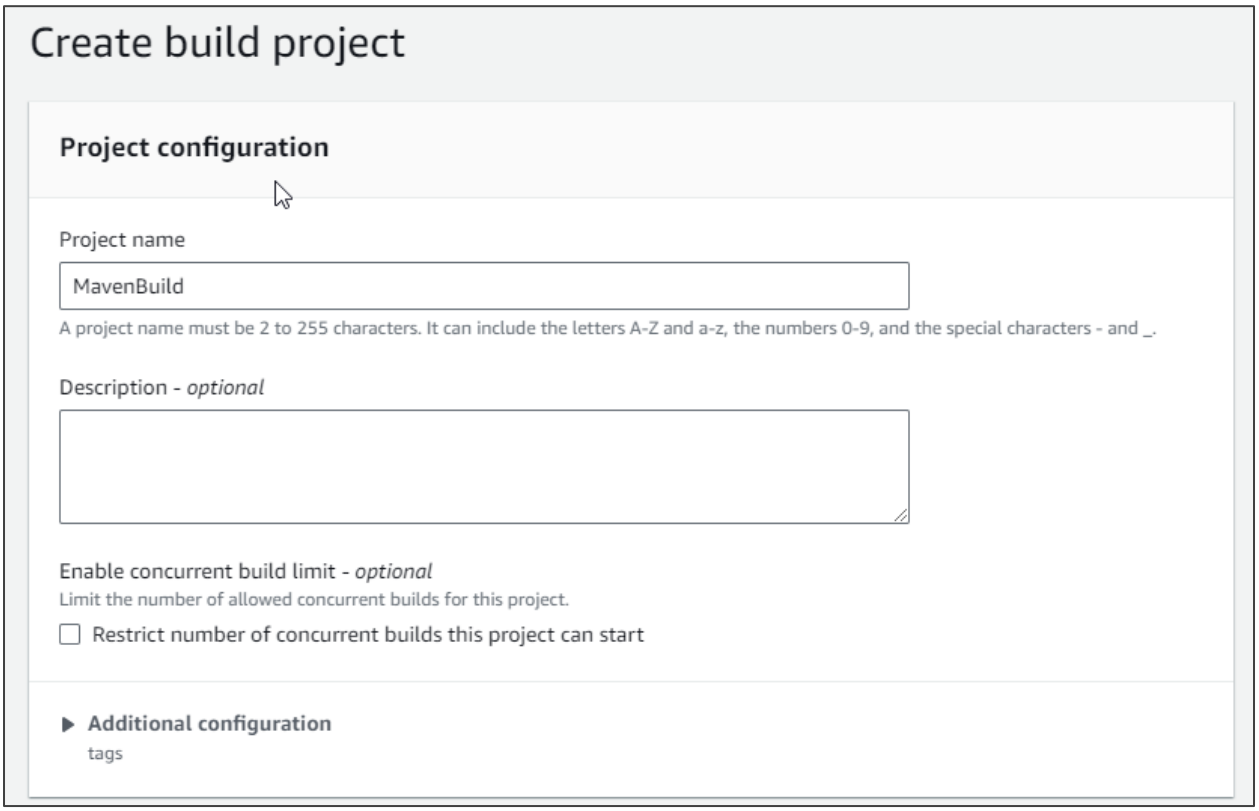
2.7 Select **AWS CodeBuild** as the **Build provider** for building and automating tests



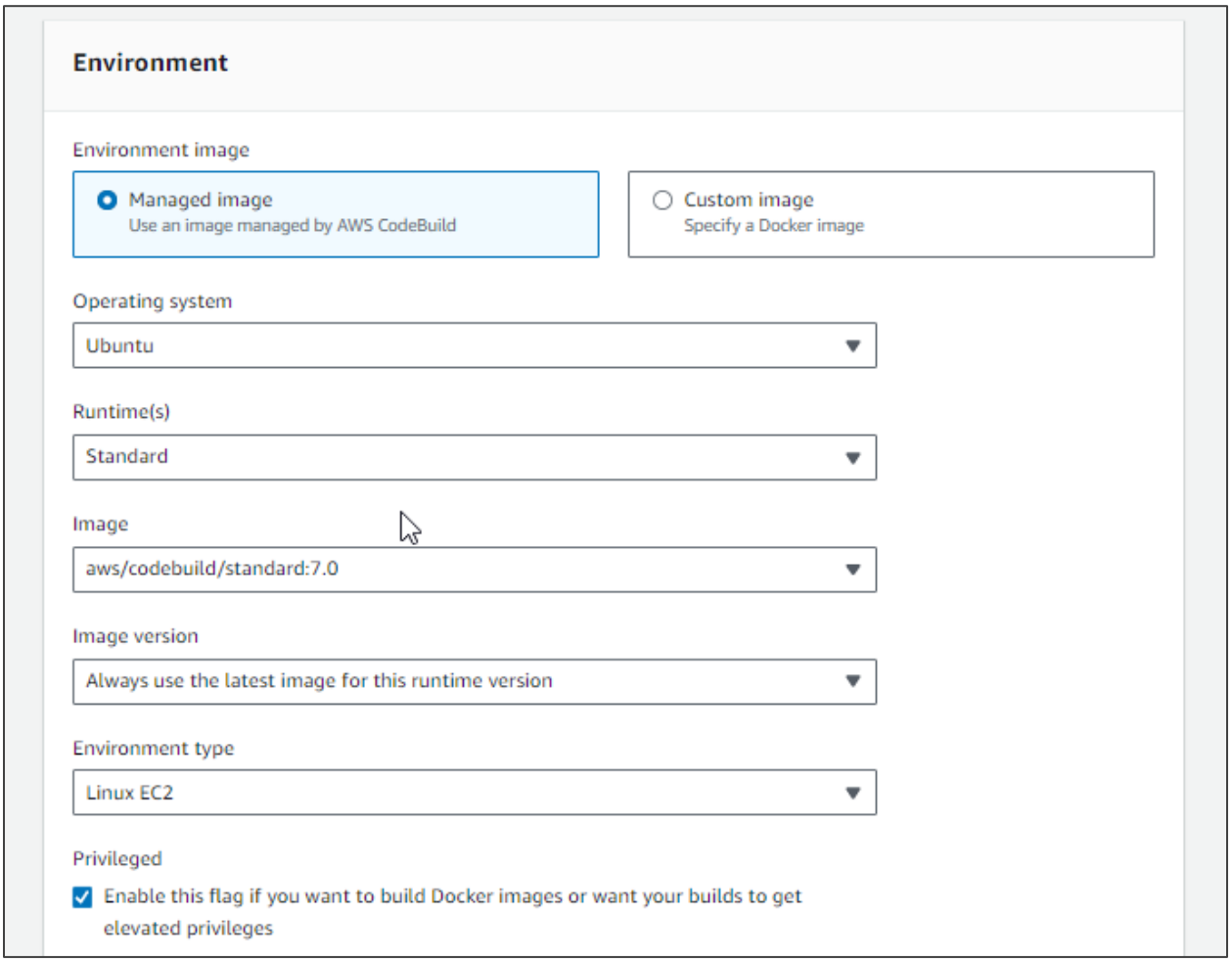
2.8 Click on the **Create project** button and a new pop-up window will appear

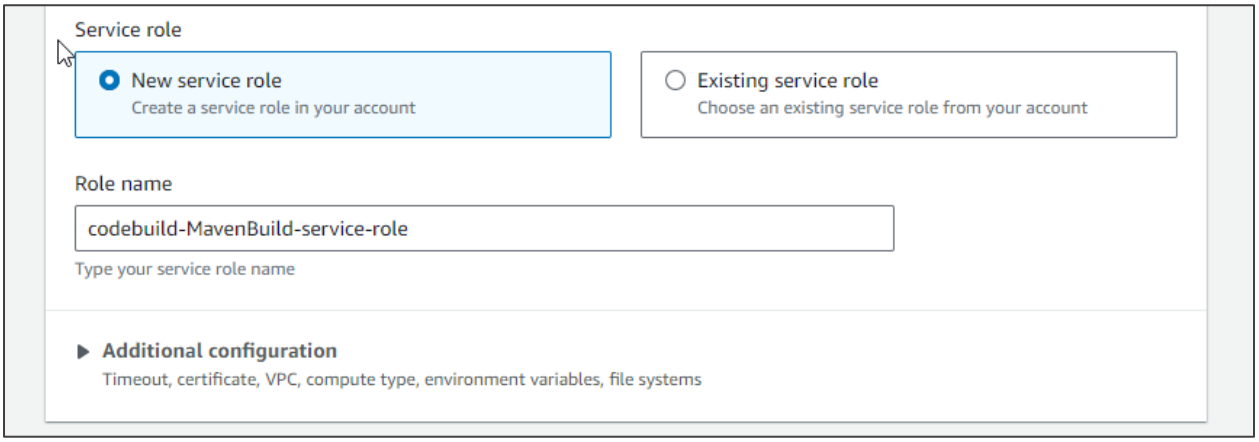


2.9 In the new window, enter the name of the project

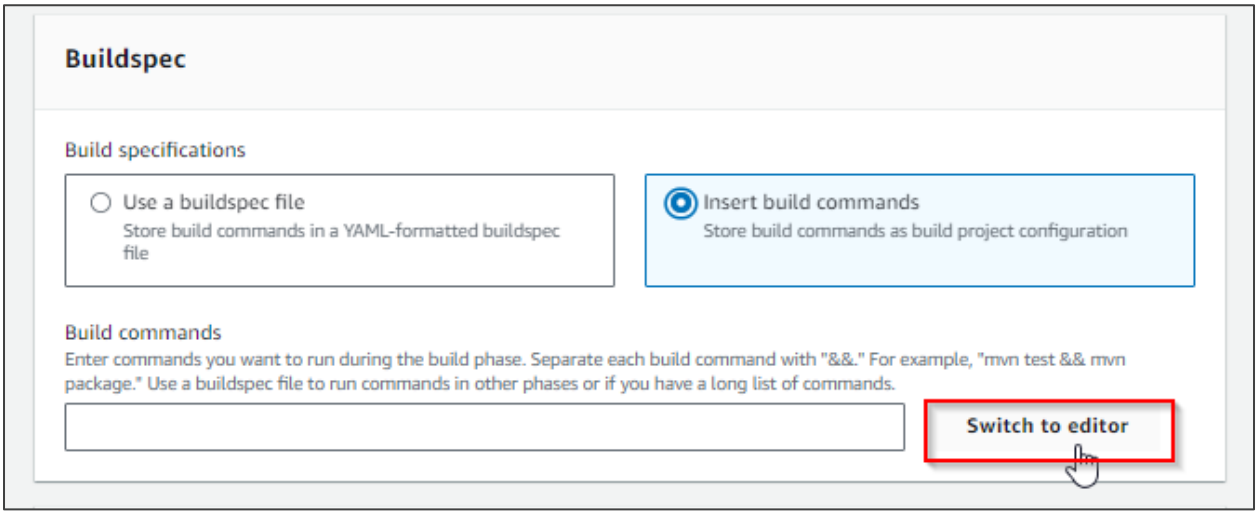


2.10 In the **Environment** section, add the details as shown in the screenshots below:





2.11 Under **Build specifications**, select the **Insert build commands** option and click on the **Switch to editor** button



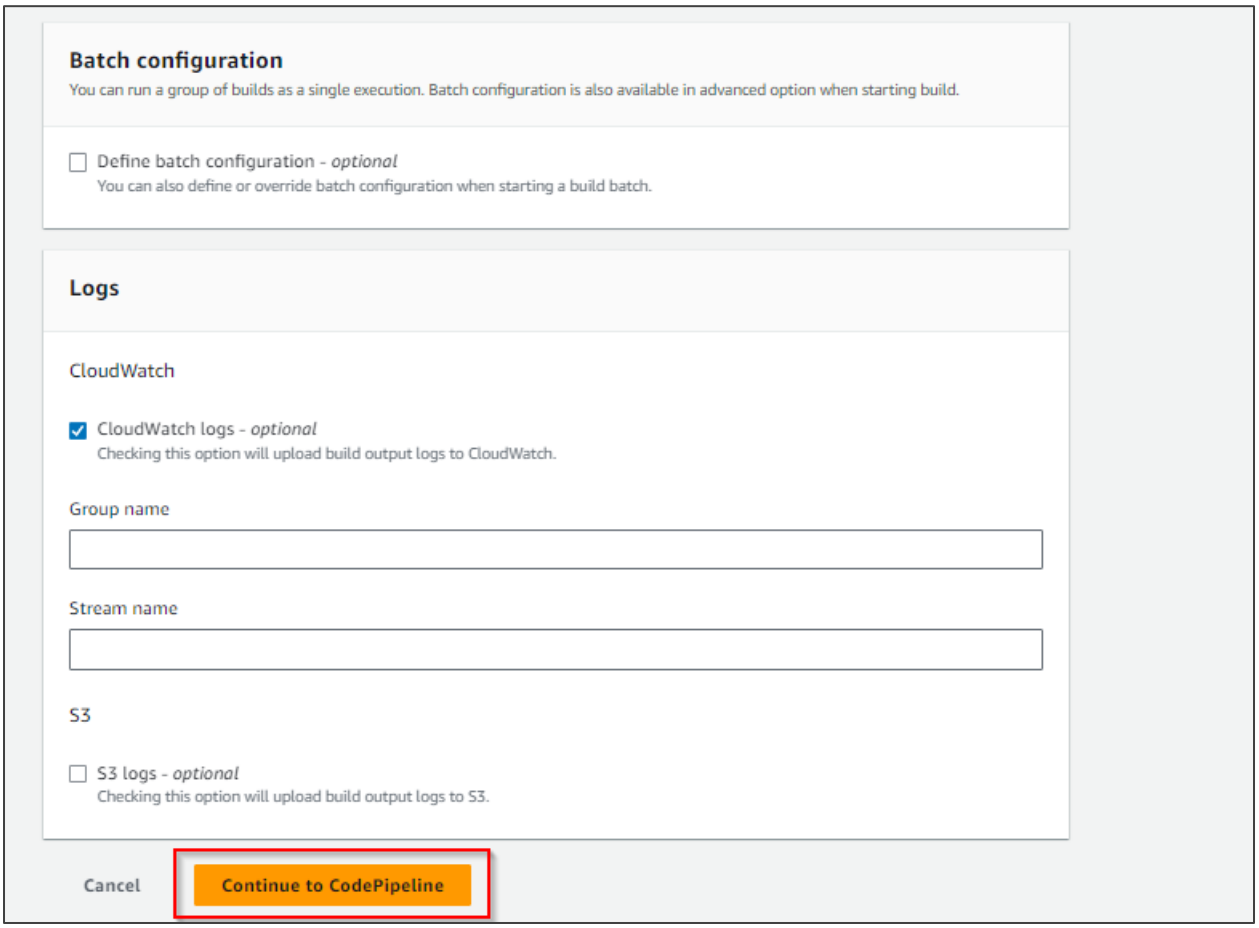
2.12 Remove the existing build commands from the editor and enter the YAML code below:

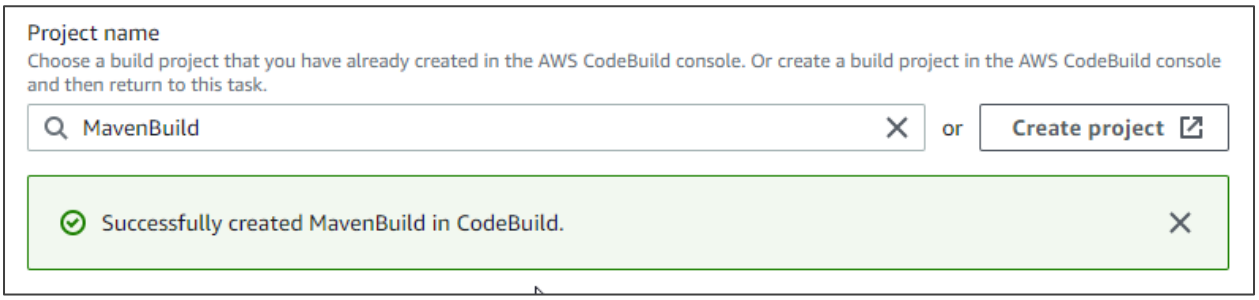
```
version: 0.2

phases:
  build:
    commands:
      - "mvn clean install"
      - "cp target/*.war ROOT.war"
artifacts:
  files:
    - "ROOT.war"

  name: $(date +%Y-%m-%d)
  discard-paths: yes
  base-directory: .
```

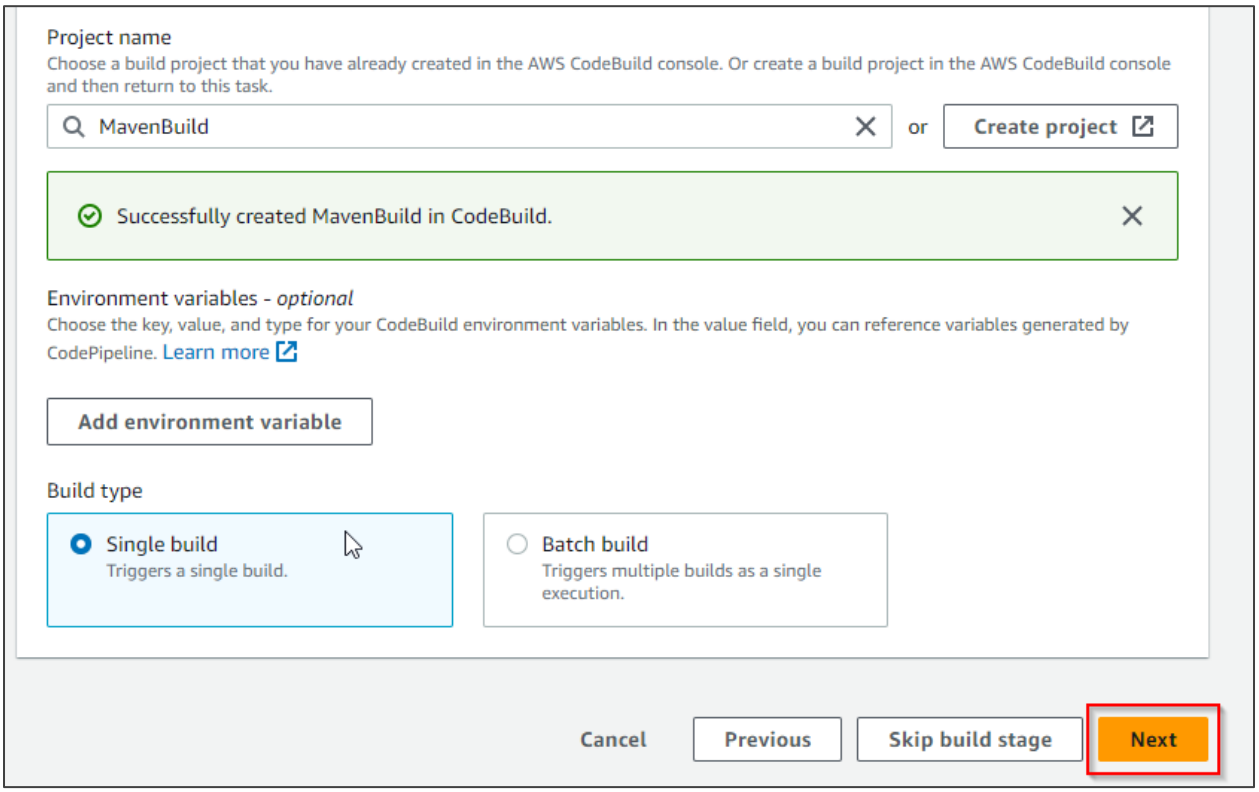
2.13 Scroll down to the bottom of the page and click on **Continue to CodePipeline**





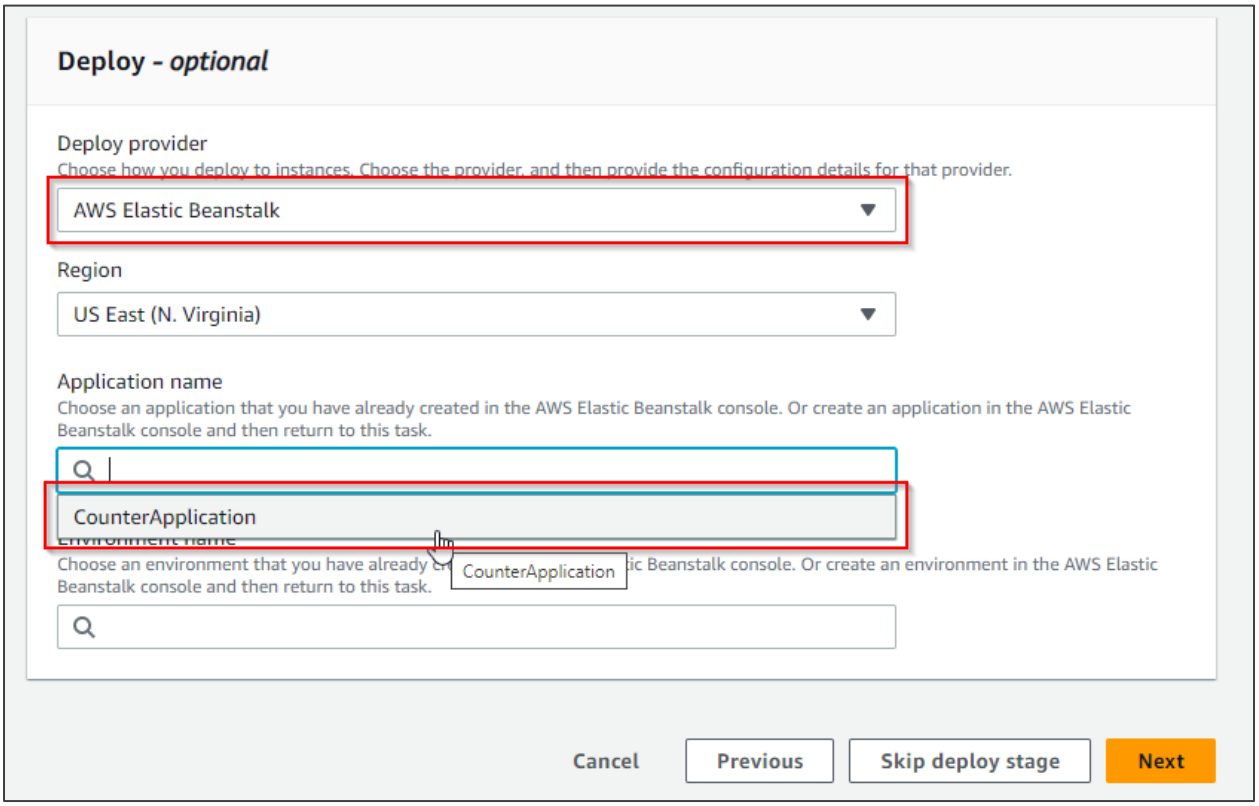
The project is successfully created.

2.14 Now, click the **Next** button

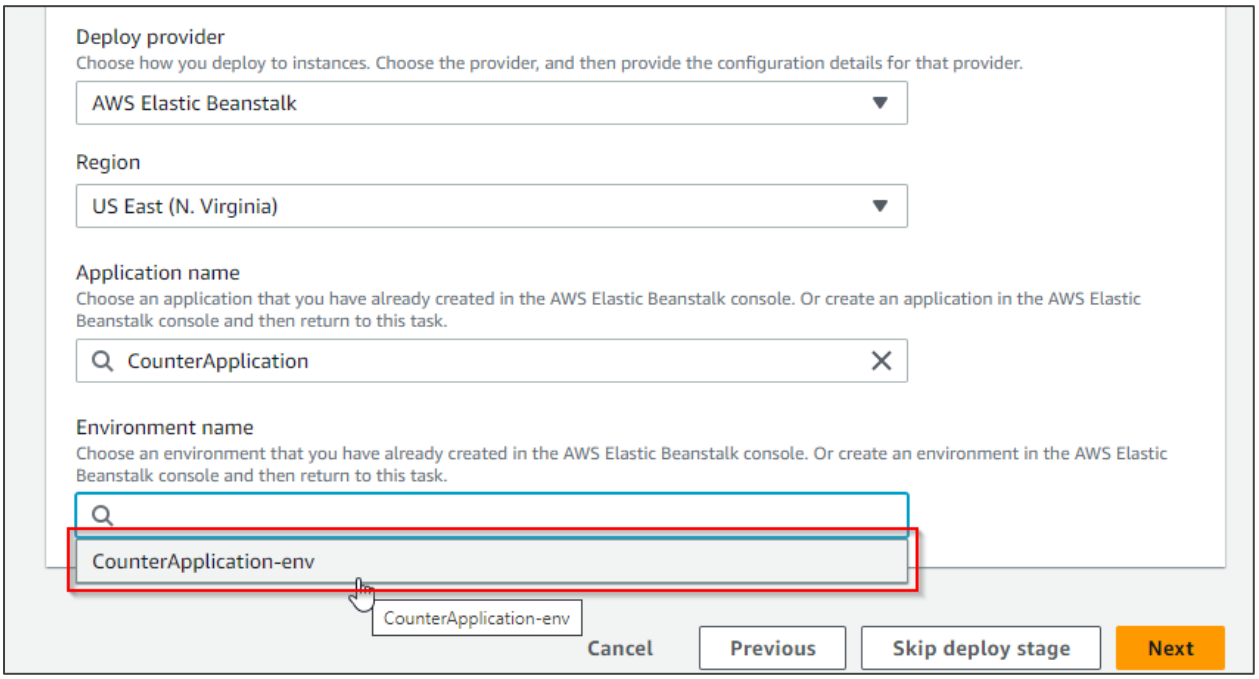


2.15 Select **AWS Elastic Beanstalk** as the **Deploy provider** and **CounterApplication** as the **Application name**

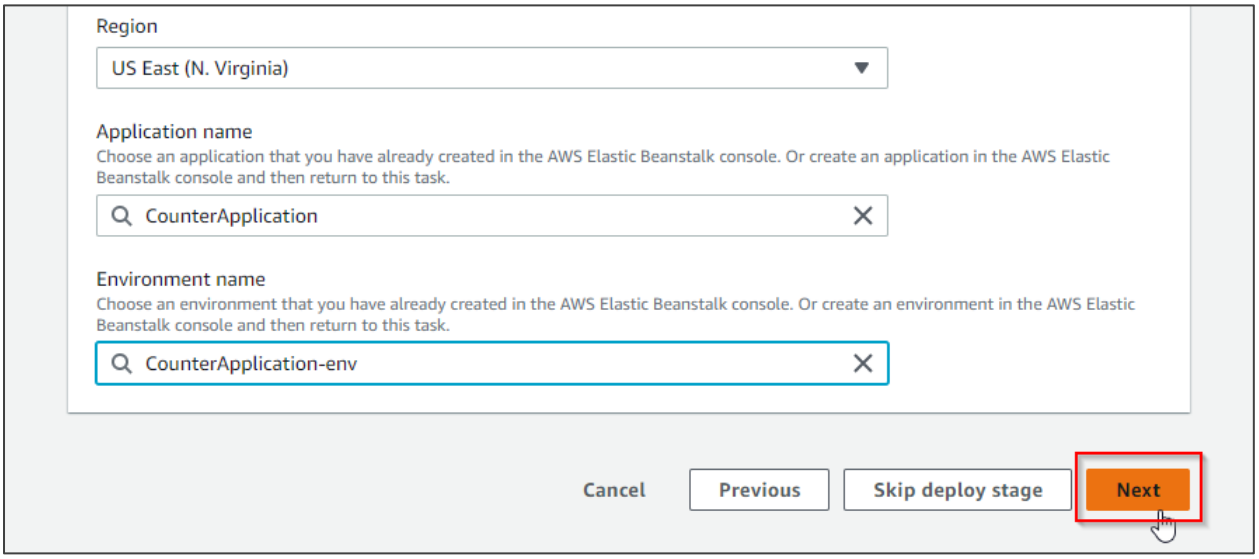
Note: Refer to the previous demo to set up a Java-based Elastic Beanstalk environment on AWS and access a sample application deployed on Elastic Beanstalk



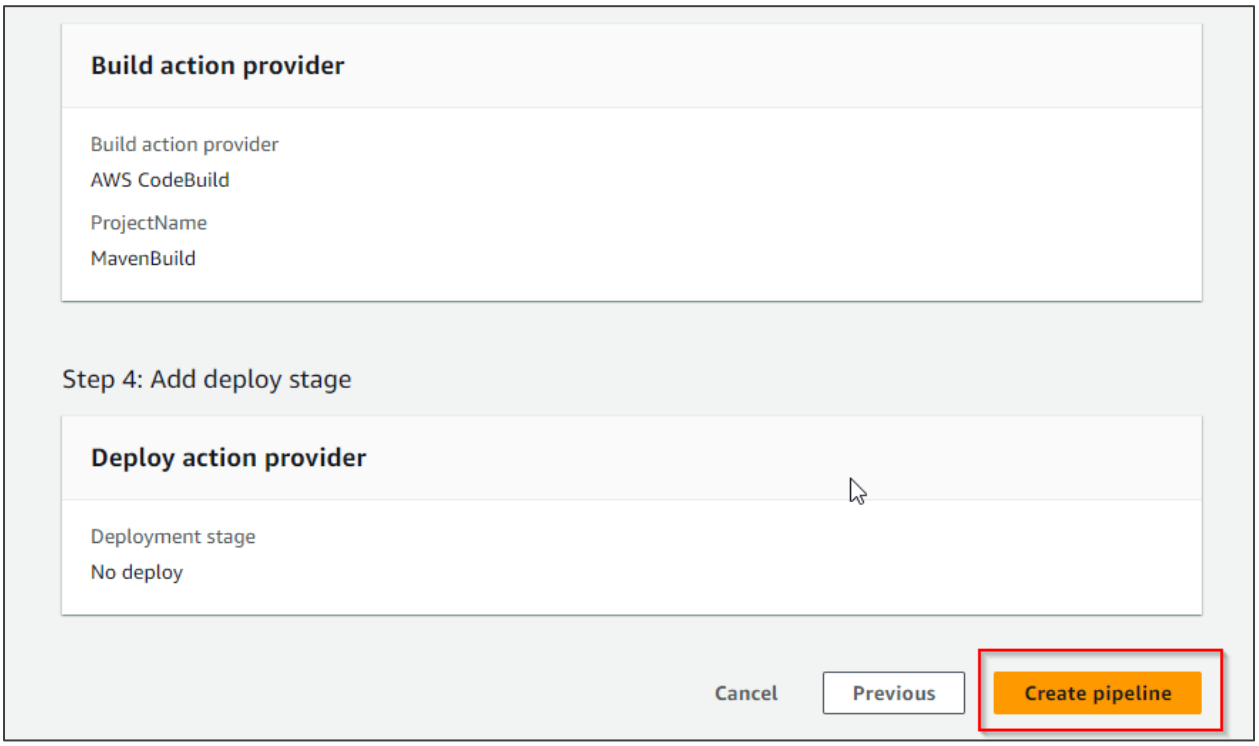
2.16 Select **CounterApplication-env** as the **Environment name**



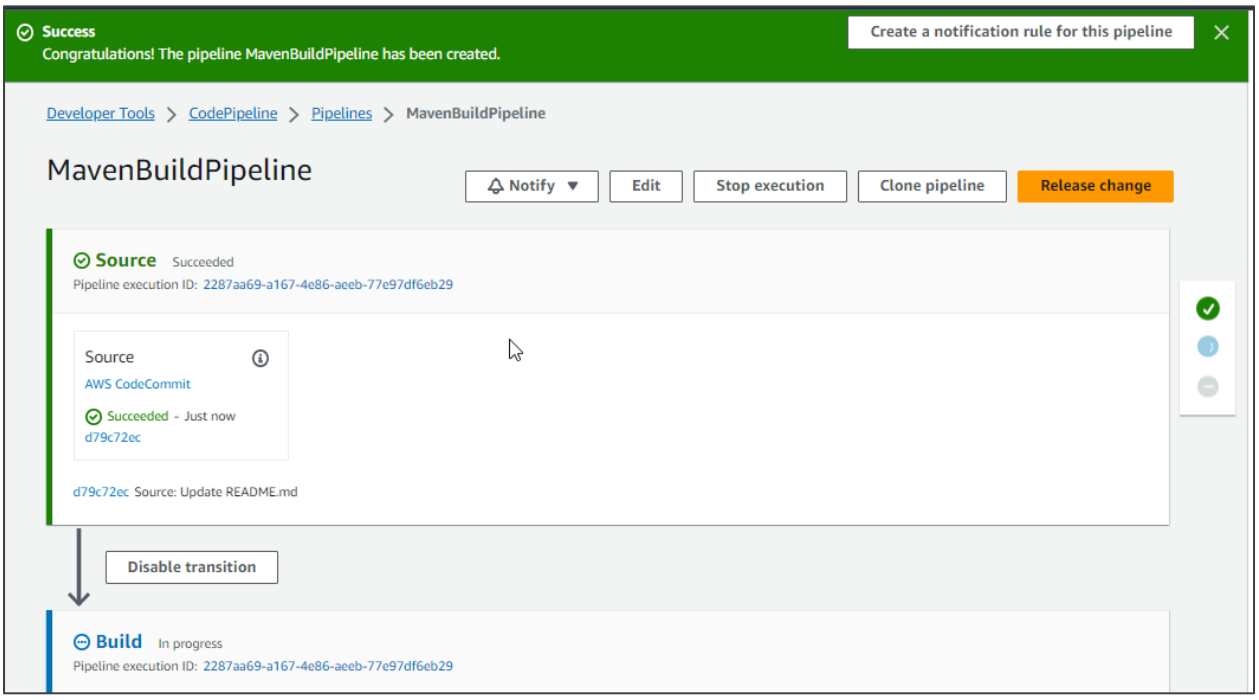
2.17 Click on the **Next** button

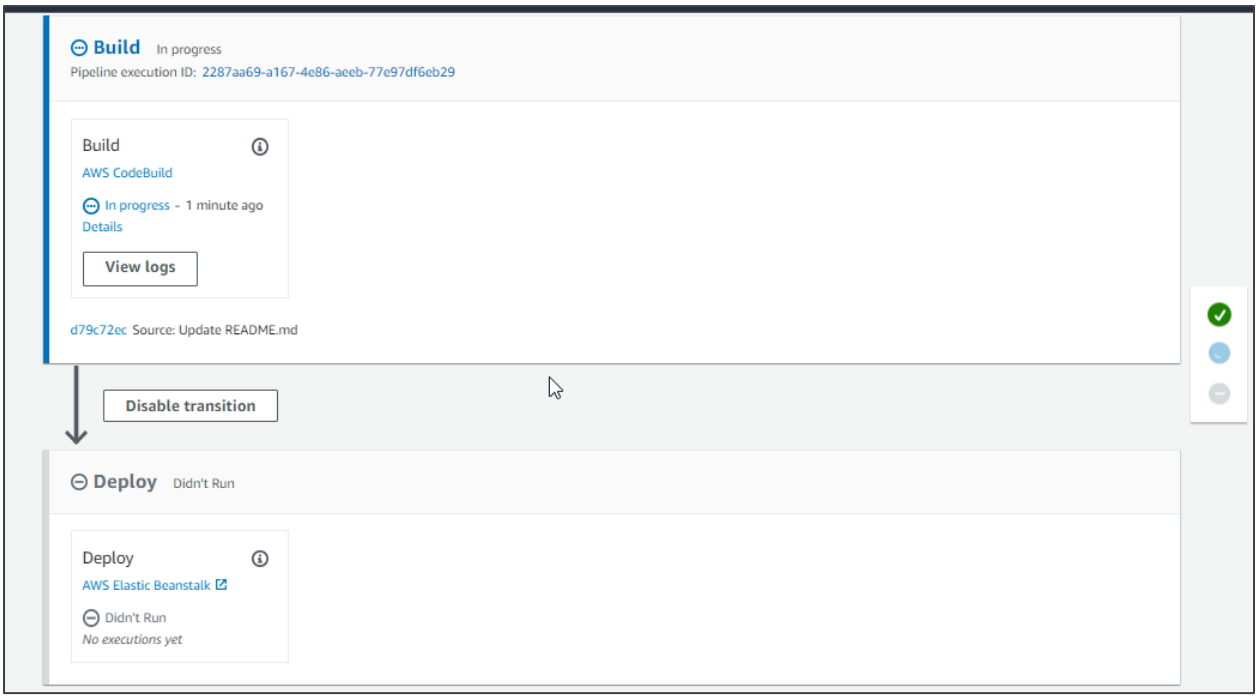


2.18 Scroll to the bottom of the page and click on the **Create pipeline** button

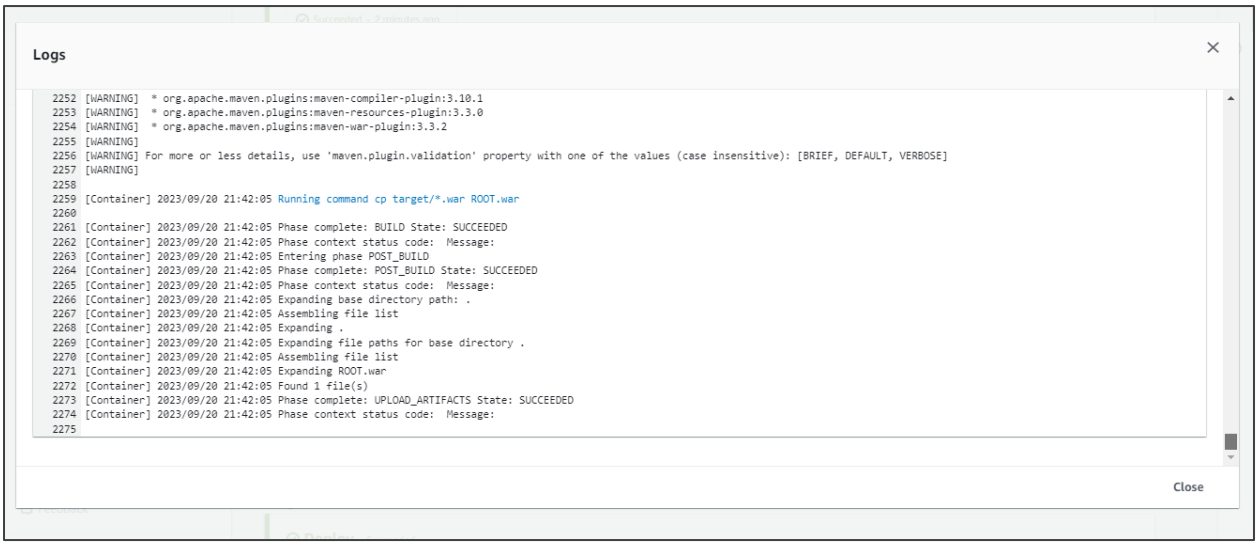


After creating the pipeline, the execution will start.



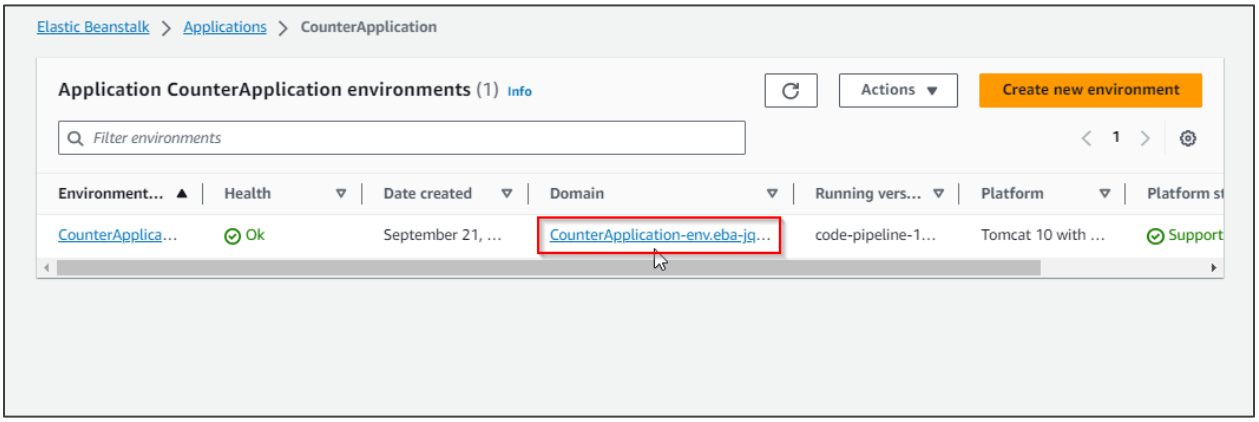


2.19 Once the build stage is completed, click on the **View logs** button in the **Build** section for validation

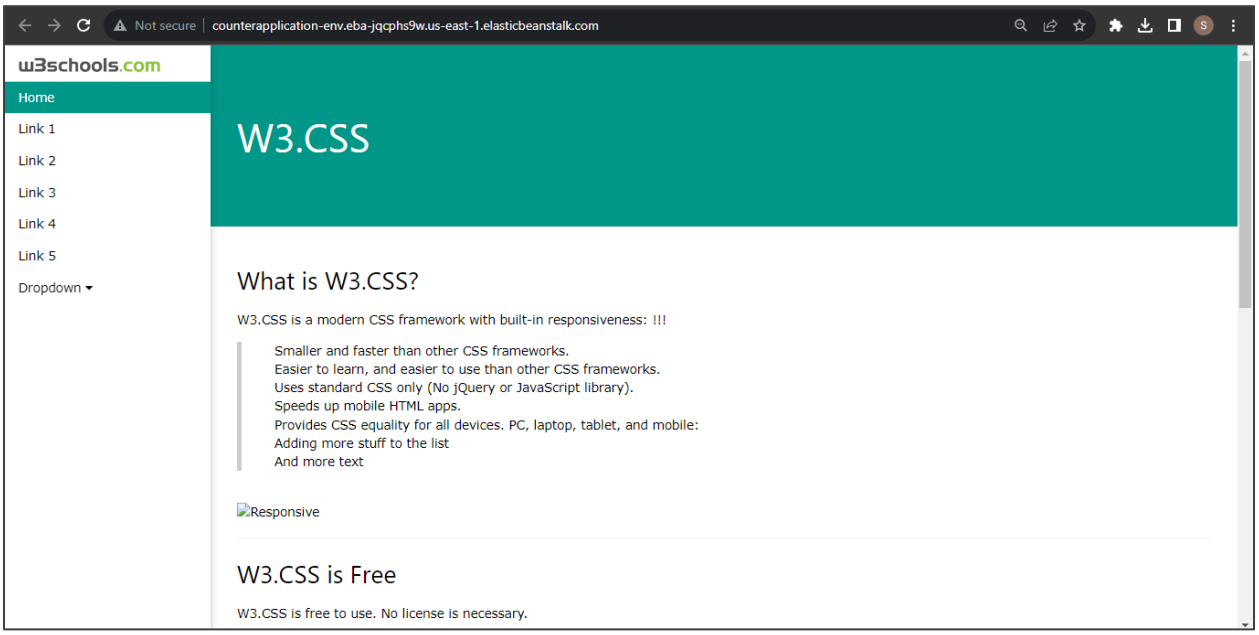


2.20 Click on the **AWS Elastic Beanstalk** link in the **Deploy** section

2.21 Now, click on the **Domain** link



You will be able to see the following interface:



By following these steps, you have successfully implemented a blue/green deployment strategy using CodeDeploy and established proficiency in setting up a continuous integration pipeline with CodePipeline and CodeBuild to automate Maven-based build and test processes.